

LAB5 REPORT

Name: Mourya Nandan

Roll No: CS23B006

QUESTION 1:

What we want to do:

- The goal of this assignment was to **implement Copy-on-Write (COW) fork** in xv6.
In the original xv6, when a process calls `fork()`, the kernel copies the entire address space of the parent into the child. This is wasteful, because often the child immediately calls `exec()` or only reads from those pages.
- We want to optimize `fork()` so that:
- Instead of duplicating all memory immediately, **both parent and child share the same physical pages**.
- These shared pages are marked **read-only and Copy-on-Write (COW)**.
- If either process later tries to write to a shared page, the kernel makes a **private copy of the page** at that moment.
- This saves memory and reduces overhead.

What I Have Done:

kalloc.c – Managing Physical Pages

Before (original xv6 behavior)

- xv6 managed physical memory in a free list (`kmem.freelist`).
- `kalloc()` pulled one free page and returned it.
- `kfree()` put the page back to freelist.
- There was no concept of sharing pages or tracking references.

So whenever `fork()` ran, every page was duplicated, and freeing was straightforward because each page belonged to exactly one process.

After (with COW modification)

Added Reference Counter

```
static int refcnt[(PHYSTOP - (uint64)end) / PGSIZE];
```

```
struct spinlock ref_lock;
```

- We introduced `refcnt[]` to track how many processes share a page.
- This is essential because now a physical page may belong to multiple processes (parent and child).

Modified kalloc()

```

void *
kalloc(void)
{
    struct run *r;

    acquire(&kmem.lock);

    r = kmem.freelist;

    if(r)
        kmem.freelist = r->next;

    release(&kmem.lock);

    if(r) {
        memset((char*)r, 5, PGSIZE);

        int idx = PA2IDX((uint64)r);

        acquire(&ref_lock);

        refcnt[idx] = 1; // fresh page starts with 1 reference

        release(&ref_lock);
    }

    return (void*)r;
}

```

Explanation:

- Originally kalloc() just handed out a new page.
- Now, every time a page is allocated, we set its reference count = 1.
- Why? Because initially, only one process owns it. Later, fork() may make others share it.

Execution flow:

- On RISC-V, when the kernel needs memory for a process (stack, heap, trapframe), kalloc() runs.
- Our version ensures metadata (refcnt) matches hardware reality.

Modified kfree()

```

void
kfree(void *pa)
{
    uint64 pa64 = (uint64)pa;
    int idx = PA2IDX(pa64);

    if((pa64 % PGSIZE) != 0 || (char*)pa < end || pa64 >= PHYSTOP)
        panic("kfree");

    acquire(&ref_lock);
    if(refcnt[idx] < 1)
        panic("kfree: refcount already 0 or corrupted");
    refcnt[idx]--;
    if(refcnt[idx] > 0){
        release(&ref_lock);
        return; // still shared → don't free
    }
    release(&ref_lock);

    // really free only if refcount == 0
    memset(pa, 1, PGSIZE);
    struct run *r = (struct run*)pa;
    acquire(&kmem.lock);
    r->next = kmem.freelist;
    kmem.freelist = r;
    release(&kmem.lock);
}

```

Explanation:

- Before: every call to `kfree()` would immediately free the page.
- After: we decrement the reference count.
 - If other processes still reference the page (`refcnt > 0`), we don't free.
 - Only when the last reference disappears do we put the page back to freelist.

Execution flow:

- When `exit()` or `munmap()` unmaps a page, `kfree()` is called.
- On RISC-V, instead of blindly freeing, our logic checks whether the physical page is still used elsewhere.

vm.c – Virtual Memory Management

Original `uvmcopy()`

```
int
uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
{
    for(i = 0; i < sz; i += PGSIZE){
        pa = walkaddr(old, i);
        mem = kalloc();
        memmove(mem, (char*)pa, PGSIZE);
        mappages(new, i, PGSIZE, (uint64)mem, flags);
    }
    return 0;
}
```

- Before: for every virtual page in parent, allocate new memory and copy contents into the child.
- This is why fork was expensive.

Modified `uvmcopy()`

```
int
uvmcopy(pagetable_t old, pagetable_t new, uint64 sz)
{
    pte_t *pte;
```

```

uint64 pa, i;

uint flags;

for(i = 0; i < sz; i += PGSIZE){
    if((pte = walk(old, i, 0)) == 0)
        panic("uvmcopy: pte should exist");
    if((*pte & PTE_V) == 0)
        panic("uvmcopy: page not present");

    pa = PTE2PA(*pte);
    flags = PTE_FLAGS(*pte);

    // remove write, mark COW
    flags = (flags & ~PTE_W) | PTE_COW;

    // map into child with same pa
    if(mappages(new, i, PGSIZE, pa, flags) != 0)
        goto err;

    incref(pa); // increase refcount
    *pte = (*pte & ~PTE_W) | PTE_COW; // parent also marked COW
}

return 0;

err:

uvmunmap(new, 0, i / PGSIZE, 1);

return -1;
}

```

Explanation:

- Before: every page was copied eagerly.
- Now:
 - Parent and child share the same physical page.
 - Both PTEs are changed to read-only (~PTE_W) and tagged with PTE_COW.
 - Reference count incremented.

Execution flow:

- On RISC-V, after fork(), parent and child both point to the same PTE entry.
- When one process tries to write, the MMU traps into kernel (usertrap()), and our COW logic creates a private copy.

Original copyout()

```
int
copyout(pagetable_t pagetable, uint64 dstva, char *src, uint64 len)
{
    pa = walkaddr(pagetable, dstva);
    memmove((void*)pa, src, len);
    return 0;
}
```

- Before: It simply copied data from kernel buffer into user space at dstva.

Modified copyout()

```
int
copyout(pagetable_t pagetable, uint64 dstva, char *src, uint64 len)
{
    uint64 va0;
    pte_t *pte;
    uint64 pa;

    while(len > 0){
        va0 = PGROUNDDOWN(dstva);
```

```
if((pte = walk(pagetable, va0, 0)) == 0)
```

```
    return -1;
```

```
    pa = PTE2PA(*pte);
```

```
    if((*pte & PTE_COW) != 0){
```

```
        // copy-on-write triggered
```

```
        char *mem = kalloc();
```

```
        if(mem == 0) return -1;
```

```
        memmove(mem, (char*)pa, PGSIZE);
```

```
        acquire(&ref_lock);
```

```
        refcnt[PA2IDX(pa)]--;
```

```
        release(&ref_lock);
```

```
        *pte = PA2PTE(mem) | (PTE_FLAGS(*pte) & ~PTE_COW) | PTE_W;
```

```
        pa = (uint64)mem;
```

```
    }
```

```
    uint64 n = PGSIZE - (dstva - va0);
```

```
    if(n > len) n = len;
```

```
    memmove((void*)(pa + (dstva - va0)), src, n);
```

```
    len -= n;
```

```
    dstva += n;
```

```
    src += n;
```

```
}
```

```
return 0;
```

}

Explanation:

- Before: copyout assumed all pages were writable.
- After:
 - If destination page is marked COW, we allocate a new writable page, copy data, update PTE, decrement old refcount.
 - This ensures first write gets a fresh page.

Execution flow:

- On RISC-V, when kernel copies syscalls arguments (like `write(fd, buf)`), `copyout()` runs.
- If page is COW, hardware trap is avoided by our software fix here.

Trap Handling (trap.c)

Before

- Any illegal access (write to readonly, invalid VA) → kernel panics or kills process.
- There was no notion of “oh, maybe this is COW, let’s fix it”.

After

- In `usertrap()`, we added:
 - Detect page fault due to writing a COW page.
 - Allocate new page, copy contents, remap with `PTE_W`.
 - Resume execution instead of killing process.

Execution flow:

- On RISC-V, CPU raises store page fault exception when trying to write to a read-only COW page.
- Kernel trap handler checks `PTE_COW`, fixes it by copying, and returns.
- To user program, it looks as if memory was always writable.

Execution Flow:

Process Creation (Setting up Page Tables)

- Code involved: `uvmfirst`, `uvmalloc`, `mappages` (unchanged)
- At system start, first user process is created. Pages are allocated by `kalloc()`, refcount for each new page is set to 1.
- Effect of our change in `kalloc.c`:
 - We added a `refcnt[]` array and set `refcnt[PA2IDX(pa)] = 1` in `kalloc()`.
 - This ensures we can later track how many processes share a physical page.

Fork – Copying Parent Memory to Child (Modified Behavior)

- Original xv6 (uvmcopy):
 - For every parent PTE, allocate a new physical page with `kalloc()`.
 - `memmove()` copies content.
 - Child PTE gets writable mapping.
- Our xv6 (modified uvmcopy):
 - Instead of copying:
 - Get parent PTE and extract physical address (`PTE2PA`).
 - Mark both parent and child PTEs as read-only (`flags &= ~PTE_W`).
 - Add the `PTE_COW` flag.
 - Map the child's virtual address to the same physical page.
 - Increment reference count: `incref(pa)`.

Code path: `fork()` → calls `uvmcopy()` → modified to enforce shared mappings with COW.

Normal Execution (Reading Shared Memory)

- Code path: normal instruction execution by CPU.
- When process reads from a shared page:
 - MMU checks PTE: valid, user, readable → allows read.
 - No trap occurs.

No extra code needed — our modified PTEs allow reads but block writes.

First Write to a COW Page

- Code path: instruction in user mode → CPU checks PTE.
- Because we cleared `PTE_W` in `uvmcopy`, the write triggers a page fault.
- Trap goes to `usertrap()` (in `trap.c`).

Trap Handling – Resolving COW Fault

- Code involved: modified `copyout()` (in `vm.c`).
- In `copyout()`, before copying user data:
 - We check if the PTE has `PTE_COW`.
 - If yes:
 1. Allocate a new page with `kalloc()`.
 2. Copy old data (`memmove`).
 3. `kfree()` the old page (decrement refcount; free only if count hits 0).
 4. Update PTE: clear `PTE_COW`, set `PTE_W`, point to new page.
 5. Execute `sfence_vma()` to refresh MMU.

This is the heart of COW: converting a shared read-only page into a private writable page.

Continuing Execution After Fault

- After fixing the PTE in `copyout()`, the kernel returns to user mode.
- CPU retries the faulting instruction.
- This time the PTE has writable bit set and points to the new physical page, so the write succeeds.

Process Exit / Memory Freeing

- Code involved: `uvmunmap()` + modified `kfree()`.
- When a process unmaps pages or exits:
 - `uvmunmap()` decides if the physical page should be freed (`do_free` flag).
 - `kfree()` is called.
 - Our modified `kfree()`:
 - Decrements `refcnt[PA2IDX(pa)]`.
 - Only adds the page back to the freelist if count reaches 0.

This ensures shared pages are not freed while still mapped in another process.

End-to-End Workflow

1. `fork()` → calls modified `uvmcopy()` → sets parent+child PTEs = read-only + COW, increments `refcount`.
2. Normal execution → reads succeed, writes cause trap.
3. Trap on write → `usertrap()` → `copyout()` sees COW → allocates new page, copies data, updates PTE, decrements `refcount`.
4. Retry instruction → succeeds, now process has its own copy.
5. Exit/unmap → `uvmunmap()` → `kfree()` decrements `refcnt`, frees only if last user.

Frequently Asked Questions (FAQ) about Our COW Implementation

How are parent and child processes sharing the same physical memory after `fork()`? Which code ensures this?

In our modified `uvmcopy()` (in `vm.c`), instead of copying data into new pages, we map the child's virtual address to the **same physical address** as the parent's.

- We remove the writable bit (`flags &= ~PTE_W`) and set a new `PTE_COW` flag.
 - We then increment the physical page's reference count (`inccref(pa)` in `kalloc.c`).
- Thus, both processes point to the same physical page but cannot write to it directly.

What happens when either parent or child tries to write to a shared page? Which function handles this case?

When a process writes to a shared read-only (COW) page, the CPU raises a **page fault**. The trap enters the kernel, and eventually, the write goes through `copyout()` in `vm.c`.

- `copyout()` checks if the PTE has the `PTE_COW` flag.

- If yes, it allocates a new page using `kalloc()`, copies the data from the old page (`memmove`), decrements the reference count of the old page, and updates the PTE with the new page's address.
- Finally, the PTE is marked writable again, and the process continues execution on its private copy.

How is memory freed correctly when multiple processes are sharing the same physical page?

We modified `kfree()` in `kalloc.c` to use a **reference counting mechanism**.

- When a process unmaps a page (via `uvmunmap()`), `kfree()` is called.
- Instead of freeing the page immediately, we decrement `refcnt[PA2IDX(pa)]`.
- Only if the count reaches 0 (no process is using the page) do we actually return it to the free list.

This prevents freeing memory still in use by another process.

Why did we clear the writable bit (PTE_W) in `uvmcopy()`? What would happen if we didn't?

Clearing `PTE_W` ensures that any write to the shared page triggers a **trap**, which is essential for COW to work.

- If we didn't clear it, both parent and child would be able to write to the same physical page without copying, corrupting each other's memory and breaking process isolation.
- By making it read-only, we guarantee that the first write will trap into the kernel, where `copyout()` can safely create a private copy.

How does the system decide whether to copy a page or not during process execution?

Copying only happens **on demand**, at the time of the first write to a shared page.

- During `fork()`, all pages are initially shared with `PTE_COW`.
- As long as both processes only read, no duplication occurs.
- When a write happens, `copyout()` checks for `PTE_COW`. Only then is a new page allocated and copied.

This **lazy allocation** is the key optimization in COW — unnecessary copies are avoided.

How do we ensure the CPU/MMU sees the updated permissions and mappings after handling a COW fault?

After modifying a PTE in `copyout()` (clearing `PTE_COW`, setting `PTE_W`, and pointing to the new page), we call `sfence_vma()` to flush the TLB.

- Without this, the CPU could keep using stale cached entries and allow writes to the old page.
- With the TLB refresh, the CPU re-reads the updated PTE and correctly enforces the new mapping.